

基础数据结构

qwaszx

2026 年 1 月 29 日

本节课目标

- 加深对于 vector、栈、队列、链表、堆等数据结构的理解
- 识别常见的可以使用这些数据结构解决的问题
- 强化数据结构思维：抽象问题与数据结构的对应

提到复杂度时默认指最坏复杂度

支持什么？

- 均摊 $O(1)$ 的末尾插入删除
- $O(1)$ 的随机访问¹
- $O(\text{size} - i)$ 加均摊 $O(1)$ 在第 i 个位置插入删除
- $O(\text{max size})$ 的空间

参考：<https://cppreference.cn/w/cpp/container/vector>

¹指任意访问某个 $a[i]$

- 迭代器失效：修改后，迭代器可能变得不可用
- 占用空间正比于历史最大 size 而不是当前 size
- clear 不会释放内存。可以用 `vector<T>().swap(a)` 来释放
- 大量 push_back 会带来很大的常数。可以使用 `resize` 一次性分配需要的长度，这样和数组基本一样快

(一种) 实现

如果 `size` 超过了当前容量，那么把当前容量加倍，重新申请空间，把所有东西都搬过去。

如果额外要求空间线性于 `size` 而不是历史最大 `size` 呢？

`size` 小于当前容量的 $1/4$ 时，把容量减半并重新分配空间（为什么是 $1/4$ 而不是 $1/2$ ？

每次重新分配后，容量约等于两倍 `size`。

触发下一次重新分配需要 $\Omega(\text{size})$ 次修改，重新分配的代价是 $O(\text{size})$ 。所以，平均每次修改的代价是 $O(1)$ 。

支持什么？

- $O(1)$ 在指定元素处插入删除单个元素/另一个链表（例如可以实现 $O(1)$ 拼接）
- $O(\text{位置})$ 随机访问
- $O(\text{size})$ 的空间

洛谷 P10466 邻值查找 / 洛谷 P1168 中位数

插入往往需要平衡树之类的数据结构，但是如果倒着来就只需要先排序然后一直删除。
如果排序可以线性，那么就能做到线性。

操作大家都知道。

显然栈 $\leq$ vector。

如果队列大小已知不超过某个上界，可以用数组模拟循环队列

$O(\text{size})$ 空间需要用链表或者双端队列实现

功能上强于 `vector` 和队列，但是实现更复杂，常数也更大。

- 均摊 $O(1)$ 头尾插入删除
- $O(1)$ 随机访问

需要开很多双端队列时，要谨慎使用 STL 的 `deque`！一个空 `deque` 可能占用几百到几千字节的空间。

更好的实现：对顶栈

设想有两个栈，其底部粘在一起。于是其整体就成为一个两端开口的双端队列。

唯一的问题出在某个栈空了还要继续删的情况。此时，我们暴力把整个序列倒出来重构即可。换句话说，我们重新把序列均分到两个栈里。

使用 `vector` 来实现栈就能支持随机访问了。如果需要很紧的空间，可以用之前提到的 $O(\text{size})$ 空间的 `vector`。

每次重构后，两个 vector 都有大概 $\text{size}/2$ 个元素。

到下次重构之前，至少有 $\text{size}/2 + |\text{size}' - \text{size}/2| = \Omega(\text{size}')$ 次操作
重构花费 $O(\text{size}')$ 的代价，所以插入删除是均摊 $O(1)$ 的

在两个 vector 中各自维护自己的前缀信息，这样可以 $O(1)$ 查询目前队列中跨过分断点的任意子区间的信息和。

可以以 $\sum_i |l_i - l_{i-1}| + |r_i - r_{i-1}|$ 的代价回答多个区间询问。在很多问题中，这个总代价往往可以是线性的。

例 1：支持双端队列操作、每次询问做背包后体积 V_i 的结果 ($O(nV)$)

例 2：“不删除双指针”，如询问最短的 $\gcd > 1$ 的区间

支持什么？

- $O(1)$ 查询最小值
- $O(\log n)$ 插入
- $O(\log n)$ 删除
- $O(\log n)$ Decrease-Key
- $O(n)$ 合并

Brodal Queue 支持

- $O(1)$ 查询最小值
- $O(1)$ 插入
- $O(\log n)$ 删除
- $O(1)$ Decrease-Key
- $O(1)$ 合并

但是实现复杂、常数很大。

算法/数据结构总是在复杂度、实际表现、实现难度、常数、功能之间做取舍。

其他一些常见的堆：左偏树、斜堆、随机堆、配对堆、二项堆、斐波那契堆

从若干个初始状态开始，每个状态可以扩展出若干个后继状态，保证扩展出来的状态的权值小于原状态的权值。每次取权值最小的状态扩展，重复若干次。

通过一定的设计，很多求前 k 大/小的问题可以变成这类问题。使用堆来维护所有目前的状态，每次取出最小的一个来扩展，注意去重。扩展形成一张图，复杂度为 $O(M \log M)$ 或 $O(k \log k + M)$ （取决于堆），其中 M 为前 k 小的点邻接的**边数**。

如果每个点的扩展方式都是相同的，并且只有常数种扩展方式，并且扩展出来的状态的权值关于原状态的权值是单调的，那么可以使用队列替换堆来做到 $O(M)$ 。

方法一： (i, j) 扩展出 $(i, j + 1)$ ，初始状态为所有 $(i, 1)$

方法二： (i, j) 扩展出 $(i + 1, j)$ 和 $(i, j + 1)$ ，但是要去重

方法三： <https://www.cnblogs.com/zkyJuruo/p/18428124>

首先处理掉 “除了新的以外都加 q ”，这可以变成全局加 q 然后新的减掉 q 。

这样就是 x 扩展出 $\lfloor p(x + tq) \rfloor - (t + 1)q$ 和 $(x + tq) - \lfloor p(x + tq) \rfloor - (t + 1)q$

符合我们用队列替换堆时的要求

常数种非负边权的最短路

BFS 扩展时, 从 x 扩展出 $O(1)$ 种固定的新状态
可以使用队列替换堆

前 k 小子集和

考虑一个 DFS 过程，枚举下一个选择的数在哪

然而这样会扩展出 $\Theta(n)$ 个状态

多叉树转二叉树：左儿子右兄弟（注意兄弟的顺序）

限制大小的前 k 小子集和

要求集合大小在 $[L, R]$ 内。

考虑怎么枚举 $\text{size} = m$ 的集合。假设选了 $a_{i_1}, \dots, a_{i_m}$, 转为枚举 $d_j = i_j - i_{j-1}$, 只需要 $\sum_j d_j \leq n - 1$ 。

初始所有 $d_j = 1$, 每次枚举下一个需要增加的 d_j 即可。

同样用儿子兄弟表示法转为二叉树

有可能与 k 短路有关联，但我不会。

有兴趣的同学可以自行研究或者一起讨论一下

由于合并的结果总是越来越大的，可以使用队列替换堆。

结论:

- Huffman 编码可以最小化 $\sum_i \text{dep}_i \cdot w_i$
- w_i 的深度大约为 $\log \frac{W}{w_i}$, 其中 $W = \sum w_i$

在**不均匀分治**（如树剖、点分治、分治 FFT 等）中，可以使用 Huffman 树来降低复杂度或常数

本节课的核心

- DFS：灵活的枚举
- 图和树：常见的模型